
Product Recognition on Store Shelves project report

Step A-B

Marco Scaramuzzi

28/09/2025

Contents

1.1	Project description	2
1.2	Step A	2
1.2.1	Task introduction	2
1.2.2	Task overview	2
1.2.3	Methodology and Implementation Details	3
1.2.3.1	High-level pipeline	3
1.2.3.2	SIFT on multiple channels	4
1.2.3.3	Main implementation components	7
1.2.4	Results	9
1.2.5	Step A task conclusions	13
1.3	Step B	14

List of Figures

1	Computer vision pipeline	3
2	Wrong detection for models 1 and 11 in scene 3 with <code>min_count=110</code>	5
3	Wrong detection for models 1 and 11 in scene 4 <code>min_count=110</code>	5
4	Scene e1 bounding boxes	9
5	Scene e2 bounding boxes	10
6	Scene e3 bounding boxes	11
7	Scene e4 bounding boxes	12
8	Scene e5 bounding boxes	13

List of Tables

1	Results of model matching	6
---	-------------------------------------	---

1.1 Project description

1.2 Step A

1.2.1 Task introduction

The task consists in developing a pipeline for the detection of products on store shelves using computer vision techniques. The main steps include loading images, extracting features and descriptors, matching descriptors between model and scene images and estimating the position of products in the scene.

The notebook `step_A.ipynb` contains the runnable pipeline; this report highlights the chosen methodology and implementation details, results and a short conclusion. For Step A, all models were correctly recognized.

1.2.2 Task overview

The task A requires to implement a single-instance product detector that localizes cereal boxes in shelf images. The task was attempted using SIFT features, computed independently on the RGB channels, FLANN matching with Lowe's ratio test, and homography estimation with RANSAC.

The processing pipeline is compactly represented in Fig. 1.

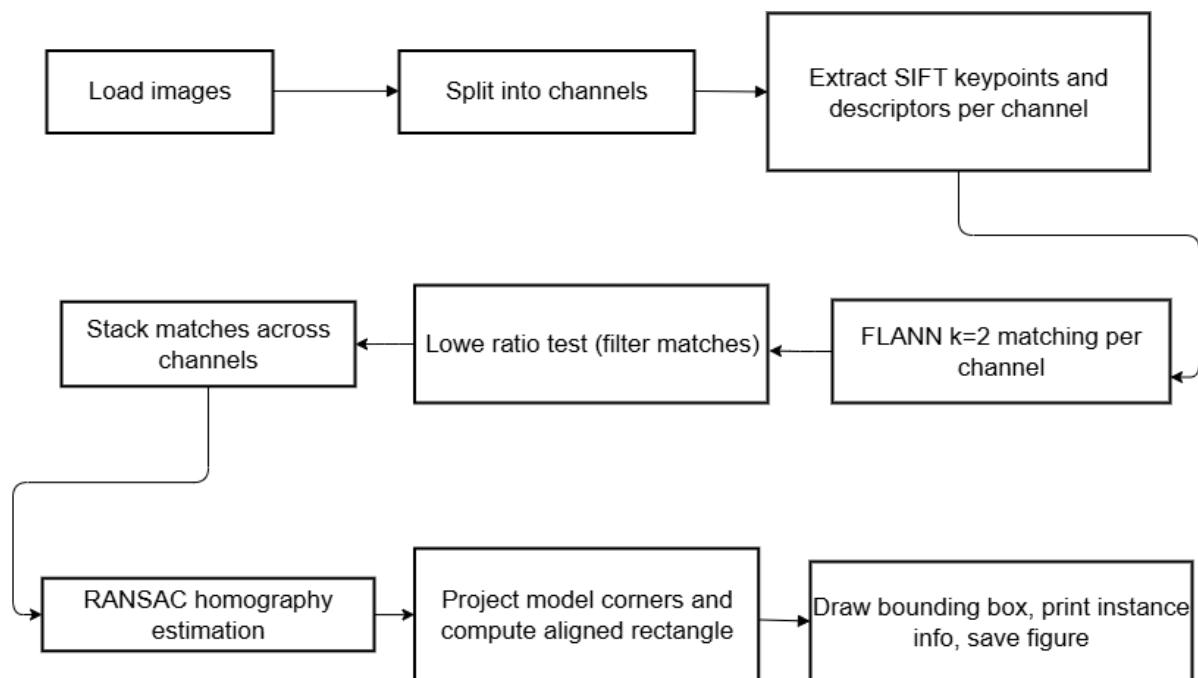


Figure 1: Computer vision pipeline

1.2.3 Methodology and Implementation Details

This section gives a deeper explanation of the pipeline (both theoretical motivations and implementation details). We will first look at the high-level steps of the pipeline, then we will discuss the motivations for using SIFT on multiple channels, and finally we will look at the main implementation components.

1.2.3.1 High-level pipeline

The main steps of the pipeline are as follows:

1. **Per-channel feature extraction:** Each model and scene image is split into R/G/B channels (function `create_channel_dict`) and SIFT keypoints/descriptors are computed independently on each channel using `cv2.SIFT_create()` inside `extract_features_dict`. The result is two nested dictionaries (`keypoints_dict` and `descriptors_dict`) indexed first by image id and subsequently by color channel.
2. **Channel-wise matching:** For each model-vs-scene pair, descriptors from color channel `c` of the model are matched against descriptors from channel `c` of the scene using a FLANN approximate k-NN search ($k=2$). The reason for $k=2$ is to enable Lowe's ratio test which requires the nearest and second-nearest neighbors. The matching is implemented in the `compute_matches_dict_single` function.

3. **Ratio-based filtering:** The `filter_matches` function applies Lowe's ratio test, which is expressed as the formula $\text{dist}_1 < 0.7 \times \text{dist}_2$. Here, dist_1 represents the distance to the nearest neighbor, and dist_2 represents the distance to the second-nearest neighbor in the descriptor space. This test removes ambiguous matches where the nearest neighbor is not sufficiently better than the second nearest. This filtering is applied inside the `compute_good_matches_dict_single` function, which returns only the best matches per channel.
4. **Aggregation and geometric verification:** The filtered 'good' matches from the three channels are stacked into an array of source and an array of destination points. These stacked correspondences feed `cv2.findHomography(..., cv2.RANSAC, ransacReprojThreshold=3)`. RANSAC rejects outlier matches and returns a robust 3×3 homography matrix that maps model coordinates to the scene.
5. **Projection and bounding rectangle:** Model corners are projected into the scene with `cv2.perspectiveTransform`.

All of these steps are orchestrated in the `main` function, which calls at each iteration the `object_retrieve` function, that encapsulates the aforementioned steps 2–5 for a single model-vs-scene pair and it returns the bounding box corners.

To produce a stable, axis-aligned bounding box, the `main` calls `compute_aligned_rectangle` (in `utils/bounding_box_utils.py`), which improves the quality of the predicted corners by constraining the rectangle to image bounds and then it computes width, height, and center coordinates of the bounding box. The cleaned corners are then drawn onto the scene image using `cv2.polylines` (in `process_and_draw_instances`).

Lastly, the `main` function saves the annotated output images with the computed bounding boxes, plots them and it displays textual information about detected products (the position of the bounding box center and its width/height).

1.2.3.2 SIFT on multiple channels

- **Color-preserving detection:** converting to grayscale discards chromatic edges and contrast that can be crucial for brand logos or colored patterns. Computing SIFT independently on R/G/B preserves such color-specific details.
- **Complementary matches:** features that are weak in one channel may be strong in another; stacking channel-wise good matches increases the chance of obtaining enough matches for robust homography estimation. This is particularly important in scenarios where products have distinct color features that may not be prominent in grayscale.
- **Practical trade-offs:** processing three channels multiplies descriptor storage and matching work by $\approx \times 3$. Given that the main goal of our project is to reliably detect single instances of known products in controlled shelf images, the trade-off speed vs accuracy is acceptable. Moreover,

during experimentation it was observed that using grayscale SIFT led to instability in the model detection, making the prediction unreliable.

The issue revolved around models 1 and 11, which are visually similar but differ significantly in color (one is mainly blue while the other is mainly orange). When using SIFT detector on the grayscale image, the outcome was that it struggled to differentiate between the two similar boxes and it just detected the first one it encountered, leading to incorrect matches and missed detections. Also raising the `min_count` parameter did not fully resolve the issue, because it would make the detection not reliable when models 1 and 11 were present in the same scene.



Figure 2: Wrong detection for models 1 and 11 in scene 3 with `min_count=110`



Figure 3: Wrong detection for models 1 and 11 in scene 4 `min_count=110`

For example, by inspecting Fig. 2, the grayscale approach was able to detect both models in the scene, but it identified them on the same box. In other words, it only confirmed their presence in the scene without correctly distinguishing and localizing each of them by matching them to the appropriate cereal box. This can be emphasized also by inspecting the textual output:

- Product 0 — 1 instance found:
 - Instance 1 {position: (160, 738), width: 321px, height: 435px}
- Product 1 — 1 instance found:
 - Instance 1 {position: (464, 687), width: 297px, height: 388px}

- Product 11 — 1 instance found:
 - Instance 1 {position: (464, 690), width: 303px, height: 395px}
- Product 25 — 1 instance found:
 - Instance 1 {position: (554, 220), width: 319px, height: 440px}
- Product 26 — 1 instance found:
 - Instance 1 {position: (206, 221), width: 341px, height: 442px}

We can quickly notice that the bounding box for model 1 and model 11 are almost overlapping, which is not correct.

The same issue can be observed in Fig. 3, where the grayscale approach again detected both models but failed to localize them correctly, leading to inaccurate bounding boxes.

Table 1: Results of model matching

model_id	scene_id	num_gray	min_match_count	above_threshold
0	1	163	112	51
11	1	160	112	48
24	2	196	112	84
25	2	163	112	51
26	2	237	112	125
0	3	145	112	33
0	4	138	112	26
11	4	152	112	40
25	4	149	112	37
26	4	213	112	101
19	5	170	112	58
25	5	163	112	51

See Table Table 1 for details.

In this context, the RGB approach, while more computationally intensive, provided a more robust and reliable detection.

1.2.3.3 Main implementation components

- SIFT: instantiated via `sift = cv2.SIFT_create()` and applied per-channel in `extract_features_dict`. Model images were resized to a common shape (180, 240) before feature extraction to reduce inter-model scale variability. This normalization stabilizes keypoint distribution and simplifies numeric thresholds like `min_count`.
- FLANN: the matcher is built in `initialize_flann()` with KD-tree (`trees=5`) and search `checks=50` (these parameters are tuned for a balance between speed and accuracy). FLANN's `knnMatch(..., k=2)` returns pairs used by Lowe's ratio. It is particularly efficient for high-dimensional descriptor matching, leveraging approximate nearest neighbor search to balance speed and accuracy.
- Lowe ratio test: implemented in `filter_matches` with threshold 0.7 (a standard value; higher values are stricter). The function returns the two best matches per channel.
- Channel-wise matching and per-channel ratio filtering: for each model–scene image pair, descriptors are matched independently on each color channel using FLANN's k-NN search with `k=2` (nearest and second-nearest neighbors). Lowe's ratio test is applied separately on each channel to filter ambiguous correspondences and retain only high-confidence matches per channel. The retained per-channel matches are then converted to coordinate arrays and concatenated across channels to form the source and destination point sets that feed the geometric verification stage.
-
- RANSAC/Homography: `compute_homography` stacks matched keypoint coordinates and calls `cv2.findHomography(..., cv2.RANSAC, ransacReprojThreshold=3)` to estimate a robust transformation. The reprojection threshold controls inlier acceptance and was set to a stricter value than the default (5) to ensure higher-quality bounding box estimation.

Listing 1: Data structures for keypoints and descriptors

```

1 keypoints_dict = {
2     model_indexes[0]: {
3         'R': [<cv2.KeyPoint 0x1>, <cv2.KeyPoint 0x2>],
4         'G': [<cv2.KeyPoint 0x3>, <cv2.KeyPoint 0x4>],
5         'B': [<cv2.KeyPoint 0x5>, <cv2.KeyPoint 0x6>]
6     },
7     model_indexes[1]: {
8         'R': [<cv2.KeyPoint 0x7>, <cv2.KeyPoint 0x8>],
9         'G': [<cv2.KeyPoint 0x9>, <cv2.KeyPoint 0x10>],
10        'B': [<cv2.KeyPoint 0x11>, <cv2.KeyPoint 0x12>]
11    }
12    # ... more keys for each image id
13 }
```

The structure of `descriptors_dict` is similar to `keypoints_dict`, the inner dictionaries values

are numpy arrays instead of list of keypoints.

1.2.4 Results

In this section are reported the detections produced by computer vision pipeline applied to the five scenes. The results will be structured both as images and textually.

The provided images represent the bounding boxes of the detected models and the model number drawn on each scene image.

The textual output reports the number of detected products instances alongside the width/height and position of the center of the bounding box.

- Scene e1 — visualization

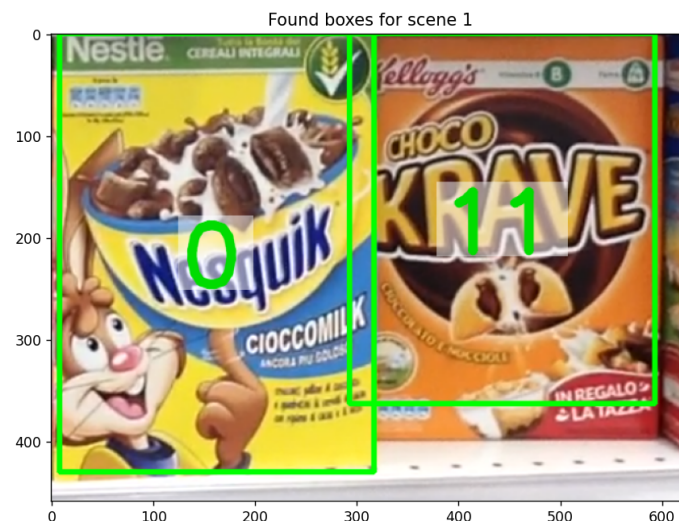


Figure 4: Scene e1 bounding boxes

- Product 0 — 1 instance found:
 - * Instance 1 {position: (162, 215), width: 309px, height: 430px}
- Product 11 — 1 instance found:
 - * Instance 1 {position: (444, 180), width: 299px, height: 361px}

- Scene e2 — visualization



Figure 5: Scene e2 bounding boxes

- Product 24 — 1 instance found:
 - * Instance 1 {position: (167, 232), width: 334px, height: 464px}
- Product 25 — 1 instance found:
 - * Instance 1 {position: (878, 232), width: 312px, height: 440px}
- Product 26 — 1 instance found:
 - * Instance 1 {position: (538, 230), width: 333px, height: 461px}

- Scene e3 — visualization



Figure 6: Scene e3 bounding boxes

- Product 0 — 1 instance found:
 - * Instance 1 {position: (170, 234), width: 323px, height: 435px}
- Product 1 — 1 instance found:
 - * Instance 1 {position: (818, 198), width: 303px, height: 396px}
- Product 11 — 1 instance found:
 - * Instance 1 {position: (476, 192), width: 303px, height: 385px}

- Scene e4 — visualization



Figure 7: Scene e4 bounding boxes

- Product 0 — 1 instance found:
 - * Instance 1 {position: (160, 738), width: 320px, height: 435px}
- Product 11 — 1 instance found:
 - * Instance 1 {position: (464, 688), width: 303px, height: 395px}
- Product 25 — 1 instance found:
 - * Instance 1 {position: (554, 218), width: 319px, height: 435px}
- Product 26 — 1 instance found:
 - * Instance 1 {position: (206, 221), width: 341px, height: 442px}

- Scene e5 — visualization



Figure 8: Scene e5 bounding boxes

- Product 19 — 1 instance found:
 - * Instance 1 {position: (504, 191), width: 295px, height: 382px}
- Product 25 — 1 instance found:
 - * Instance 1 {position: (161, 228), width: 320px, height: 444px}

1.2.5 Step A task conclusions

The SIFT-per-channel + FLANN + RANSAC pipeline reliably localizes single instances of the provided product models in the five test scenes. The saved `step_a/figures/scene_*.png` images contain the annotated outputs used to verify the detections.

1.3 Step B